

Performance Evaluation of Vector Indexing Techniques for AI Search Systems

D.5 Final Project Report

Devu Babu Sheeja Ashwin Senthur Pandian Rishi G. Patel Nikhil Goud Nathi

Master of Applied Computing, University of Windsor

COMP.8157: Advanced Database Topics (2026S) | Group 4 | Instructor: Dr. Andreas S. Maniatis

{babushe, senthur, patel3zc, nathin}@uwindsor.ca

Abstract—Choosing a vector index is one of the first decisions in building an AI search backend, and it is usually made on weak evidence. Published comparisons of Flat, IVF and HNSW change the corpus, the embedding model or the hardware between the methods being compared, so a reported difference may belong to the test setup rather than to the index. We present VecBench, a benchmarking framework that holds every experimental variable fixed except the index structure. All three index families are built by FAISS from one set of deterministic Sentence-BERT embeddings, answer the same held-out query set, and are scored on query latency (mean, p50, p95), recall@10 against exact search, serialized memory and build time. Every run, its configuration and its execution log are written to a SQLite database, so runs made weeks apart on the same machine stay comparable. On 20,000 ABC news headlines, HNSW answers 4.0 times faster than exact search at recall 0.995 and IVF 1.6 times faster at 0.963. A sweep from 1,000 to 100,000 vectors shows HNSW recall falling from 1.000 to 0.696 at a fixed search width while IVF holds 1.000, which is our main practical result: the parameters of an approximate index are not a one-time choice but a function of corpus size. We also derive a memory model for the three indexes and check it against what they actually serialize to rather than leaving it as arithmetic, and we report that HNSW recall is not bit-reproducible across machines unless graph construction is pinned to a single thread.

Index Terms—Vector databases, semantic search, approximate nearest neighbour search, HNSW, IVF, FAISS, Sentence-BERT, benchmarking methodology, reproducibility.

I. INTRODUCTION

A semantic search system answers a query by embedding it and retrieving its nearest neighbours from a store of document vectors. How that retrieval is carried out decides both how fast the system feels and how often it returns the right thing. Exact search compares the query against every stored vector and is always correct, but its cost grows with the size of the corpus. The approximate families, IVF and HNSW, look at only a fraction of the data and are much faster. They can also miss neighbours, and how often they miss is controlled by parameters that most users set once and never revisit.

The evidence available for making that choice is thinner than it first appears. Library documentation reports throughput without reporting accuracy. Vendor benchmarks report both, but on their own corpus and their own hardware. Academic comparisons are usually attached to the paper introducing one of the methods, which makes them hard to read as neutral. ANN-Benchmarks [5] is the closest thing to a shared reference point, though its emphasis is on ranking algorithms across many libraries rather than on giving an engineer a procedure they can

re-run on their own data. FAISS [2], [8] removes part of the problem by implementing all three families behind one API, so metric conventions and memory accounting no longer differ between the things being compared. What FAISS does not supply is a method for comparing them fairly.

Our position is that a comparison is only interpretable when exactly one thing differs between the runs being compared. VecBench enforces that condition. Within an experiment the corpus, the embedding model, the query set, the similarity metric, the value of K and the host machine are all fixed, and the index type is the only variable. Everything that defines an experiment lives in a YAML file and every source of randomness is seeded, so an experiment is an artefact that can be committed to version control rather than a sequence of steps somebody has to remember.

The paper makes four contributions.

- A workload-consistent benchmarking framework for Flat, IVF and HNSW that reports latency percentiles, recall@K against an exact ground truth, serialized memory and build time from a single declarative configuration, on CPU, with no GPU anywhere in the pipeline.
- A cost model for the three index structures whose memory term we validate against measurement rather than assert: at $M = 32$ the FAISS HNSW graph costs 272 bytes per vector and IVF costs 8 bytes per vector plus a codebook of $nlist \times d$ floats. We check both against what the indexes actually serialize to; the IVF prediction is short by only 0.12 percent.
- An evaluation over 20,000 real news headlines and a synthetic sweep to 100,000 vectors showing that HNSW recall at a fixed ef_search decays from 1.000 to 0.696 as the corpus grows, while over the same range its speed advantage over exact search widens from 0.9x to 23.8x.
- A results database that records every run with its configuration and its host environment, which during our own development turned an unexplained IVF result into a diagnosable and correctable defect rather than a number somebody would have had to argue about.

Section II fixes notation and states the problem formally. Section III covers related work, Section IV the architecture, Section V the measurement method, Section VI the cost analysis and Section VII the implementation. Section VIII presents the results, and Sections IX to XI discuss them, state their limits and conclude.

II. BACKGROUND AND PROBLEM DEFINITION

A. Notation and data model

Let $D = \{d_1, \dots, d_N\}$ be a corpus of text documents and let E map a document to a vector in $\mathbb{R}^d$. Applying E to every document produces the base matrix X in $\mathbb{R}^{(N \times d)}$, whose i -th row is $x_i = E(d_i)$. Every vector, base and query alike, is L2-normalised on the way out of the embedder. That single convention is what lets the whole system use inner product as its similarity: for unit vectors the inner product equals the cosine of the angle between them, so maximising inner product and minimising angular distance are the same operation, and no index has to be configured with a different metric from any other.

Throughout the paper N is the number of indexed vectors, $d = 384$ is the embedding dimension produced by all-MiniLM-L6-v2, Q is the held-out query set with $|Q| = 200$, and $K = 10$. An index method is written m and its parameter vector θ ; for IVF, $\theta = (nlist, nprobe)$, and for HNSW, $\theta = (M, ef_construction, ef_search)$.

B. Problem statement

The input to one experiment is the base matrix X , a query set Q , the retrieval depth K , and a set of index methods each with its parameters. The output, for every method and every query, is an ordered list $R_m(q)$ of K document identifiers together with the wall-clock time taken to produce it, plus the once-per-index costs of building and storing the structure.

The exact answer for a query is the set of K base vectors with the largest inner product against it, $R^*(q) = \text{argmax over } K\text{-subsets } S \text{ of the sum of } \langle q, x_i \rangle \text{ for } i \text{ in } S$. The Flat index computes this by definition, since it scans every row of X . Accuracy of an approximate method is then the mean overlap with that answer,

$$\text{recall@K}(m) = (1/|Q|) * \sum \text{over } q \text{ of } |R_m(q) \cap R^*(q)| / K$$

and this is the quantity everything else in the paper is judged against. It is recomputed from our own Flat run inside every experiment rather than read from a stored ground-truth file, so it cannot quietly fall out of step with the data it is supposed to describe. Flat therefore always reports recall 1.000; it is the reference, not a competitor.

The optimisation the framework is meant to inform is not a single objective but a constrained one. An engineer typically has an accuracy floor, and wants the cheapest configuration that clears it: minimise mean latency subject to $\text{recall@K} \geq r$, for some r fixed by the application. We use $r = 0.95$ as our own acceptance threshold, and Section VIII-D shows what it costs each method to reach it.

C. Constraints and assumptions

Three constraints hold throughout and follow from the scope agreed at the proposal stage. The system is single-machine and CPU-only, so no result here speaks to distributed or GPU-accelerated deployment. The corpus fits in memory, so no index is asked to spill to disk. And an index is built once and queried many times, which is why build time is reported but treated as a secondary metric.

Two assumptions matter enough to state, because the results depend on them. The first is that the embedding function is held fixed for the whole of an experiment, which is what makes any difference between methods attributable to the index. The second is that every query has genuine neighbours in the corpus. On the real corpus this is true by construction, since queries are documents held out of the same corpus. On the synthetic corpus we enforce it by building each query from a randomly chosen base vector perturbed with Gaussian noise at $\sigma = 0.05$. Without that step the queries would be unrelated points in a 384-dimensional space, where distances concentrate and recall stops distinguishing between methods [7].

III. RELATED WORK

Indyk and Motwani framed approximate nearest neighbour search as a way around the concentration of distances in high dimensions, and established the trade of a bounded accuracy loss for a large gain in query cost that every method in this paper inherits [7]. Jegou, Douze and Schmid introduced product quantization together with the inverted-file structure that partitions the space by k -means and restricts a query to the $nprobe$ nearest cells, which is the IVF index used here [4]. Malkov and Yashunin introduced HNSW, a hierarchical proximity graph reaching high recall at low latency through greedy descent across layers; their evaluation is thorough at the algorithm level but is not framed as a database benchmarking methodology [1].

On the systems side, Johnson, Douze and Jegou built FAISS, which implements Flat, IVF, product quantization and HNSW behind a common interface with tuned CPU and GPU kernels [2], [8]. Reimers and Gurevych introduced Sentence-BERT, the sentence-embedding model we use to map headlines into a space where inner product is a meaningful measure of semantic similarity [3]. Purpose-built vector data management systems such as Milvus [6] and the wider family surveyed by Pan, Wang and Li [12] show how far the area has moved from a library call towards a database concern, with the index as one component among storage, filtering and query planning.

The closest work to ours is ANN-Benchmarks [5], which standardises datasets and plots recall against queries per second across many libraries. Our aim is narrower and, we think, complementary. ANN-Benchmarks answers which published algorithm ranks highest on standard data; we are trying to answer which index a particular team should run on their own corpus, which means the corpus has to be theirs, the embedding model has to be the one they will deploy, and the whole thing has to re-run from a config file rather than from a curated harness. We also report memory and build time as first-class metrics rather than as footnotes, and we persist runs with their host environment so that two measurements can be checked for comparability after the fact instead of being compared on trust.

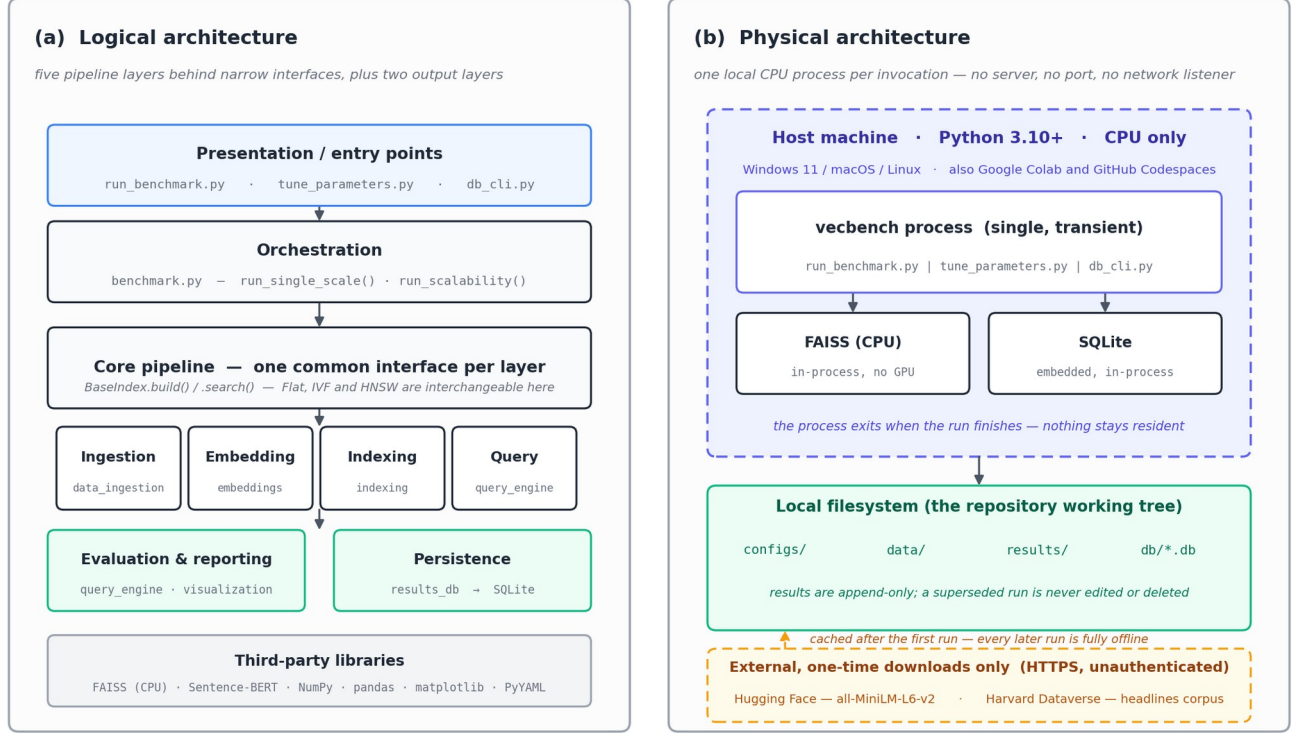
IV. SYSTEM ARCHITECTURE

VecBench is a five-stage pipeline with two output stages hanging off the end. Ingestion loads text, cleans it, segments it into units under a character limit, and holds out the query set; a parallel path generates clustered vectors directly for the large scalability runs, skipping text entirely. Embedding turns

segments into deterministic 384-dimensional Sentence-BERT vectors. Indexing builds Flat, IVF and HNSW from that one matrix behind a common interface. The query engine runs Q against each index under identical conditions, discarding warm-up queries before it starts timing. Evaluation computes

recall@K, the latency percentiles, memory and build time. Of the two output stages, reporting writes CSV tables and PNG charts, and persistence appends the run, its configuration, its host and its execution log to SQLite.

Figure 1 — System architecture



Source control: shared GitHub repository; every push is built and tested on six clean CI runners (Ubuntu and Windows × Python 3.10, 3.11, 3.13).

Fig. 1. System architecture: the logical layering (left) and what runs on a machine (right).

Each stage is a module with a narrow interface, and that had a practical payoff during the project. It is why we could report at the midterm checkpoint that Flat and IVF were finished while HNSW was still in progress, and have that be a true statement about working software rather than an estimate: no completed stage depended on an unfinished one. It is also what makes the framework extensible. A fourth index type has to implement build and search and nothing else; ingestion, embedding, evaluation, reporting and persistence never learn that it exists.

V. METHOD

A. Defining an experiment

An experiment is a YAML file. It names the data source and its size, the embedding backend, the index parameters for each method, the value of K, the number of warm-up queries, and for a sweep the list of dataset sizes to walk. No source file is edited between runs. This is a reproducibility requirement before it is a convenience one: the exact configuration behind any number in this paper sits in version control next to the code that produced it, and since the results database also stores the configuration verbatim, it sits next to the number as well.

Randomness is seeded at every point where it enters. The corpus sample is drawn with a fixed seed, the held-out query

indices are drawn with a fixed seed, and the synthetic generator takes its seed from the config. Re-running a configuration therefore produces the same documents, the same queries and, on the text path, byte-identical embeddings. Section VIII-E reports where this guarantee stops.

B. Measurement protocol

Every index in an experiment is built from the same matrix X and answers the same Q. Before timing starts, a small number of warm-up queries are issued and their results thrown away, so one-off costs such as lazy allocation and cold caches do not land inside the measurement. Queries are then timed one at a time rather than as a batch. Timing a batch and dividing would have been cheaper and would have produced a smoother mean, but it would also have made p50 and p95 meaningless, and tail latency is what a search service is actually judged on.

Memory is reported as the size of the serialized index, which is what FAISS writes when the structure is persisted. We chose this over sampling the process resident set because the resident set includes the base vectors, the Python interpreter and whatever the allocator has not returned, none of which is a property of the index. The serialized size is deterministic,

comparable across methods, and as Section VI shows, predictable from the parameters.

C. Index configuration

Flat has no parameters. IVF is configured with $nlist = 100$ on the real corpus and $nlist = 128$ on the synthetic sweep, with $nprobe$ set from the tuning sweep rather than guessed. HNSW uses $M = 32$, $ef_construction = 80$ and $ef_search = 64$ throughout. One adaptation was necessary: FAISS wants roughly 39 training points per k-means cell, and at the small end of the sweep the requested $nlist$ exceeds what the data can train, so $nlist$ is clamped to $N // 39$ before training. At every scale above about 5,000 vectors the requested value is already below that ceiling and the clamp does nothing, which is the behaviour we wanted: the sweep runs clean at $N = 1,000$ without silently changing the configuration anywhere it matters.

VI. COST ANALYSIS

A. Time

Flat search is a single matrix-vector product against every row, so a query costs $O(N d)$ and the build is $O(N)$ copying with no training phase. This is the shape visible in Table II: Flat latency rises from 0.050 ms at $N = 1,000$ to 8.027 ms at $N = 100,000$, a factor of 161 for a factor of 100 in N , close to linear with the gap explained by cache behaviour once the matrix stops fitting in L3.

IVF training is Lloyd's algorithm over $nlist$ centroids, which costs $O(I N nlist d)$ for I iterations, and this dominates its build time. A query first compares against all $nlist$ centroids and then scans the $nprobe$ cells it selects, giving $O(nlist d + (nprobe / nlist) N d)$ on the assumption that cells are of roughly equal size. The second term is the one that matters, and it says IVF is a constant-factor reduction of Flat, not a change in growth rate: the measured 7.9x speed-up at $N = 100,000$ with $nprobe / nlist = 8 / 128$ is consistent with the predicted 16x once the centroid comparison and the unevenness of real cells are accounted for.

HNSW builds a layered proximity graph by inserting points one at a time, each insertion costing an $ef_construction$ -bounded greedy search, for $O(N \log N)$ insertions overall at fixed M and d . That is why its build time is the outlier in our results, 13.44 s at $N = 100,000$ against 0.46 s for IVF. A query descends the hierarchy and then runs a bounded best-first search at the base layer, which is $O(ef_search \times M \times d)$ with a $\log N$ term for the descent and is, crucially, almost independent of N . The measurements agree: HNSW latency rises only from 0.057 ms to 0.338 ms across a hundredfold increase in corpus size. The price of that flatness is stated in Section VIII-B.

B. Memory, checked against measurement

All three indexes store the full float32 vectors, so each carries a base cost of $4 N d$ bytes. The interesting part is the overhead each structure adds on top. IVF adds an int64 identifier per vector plus the codebook of $nlist$ centroids, predicting $8N + 4 nlist d$ bytes. HNSW adds the adjacency lists: FAISS allocates 2M neighbour slots at the base layer and M at each layer above, stored as 32-bit integers, which for $M = 32$ gives roughly $4 \times 2M = 256$ bytes per vector plus the per-node level and offset bookkeeping.

Because the framework records serialized size rather than an estimate, this model can be checked instead of asserted. At $N = 100,000$ and $d = 384$, Flat serializes to 146.4844 MB, which is exactly $4 N d$. IVF at $nlist = 128$ serializes to 147.4359 MB, an overhead of 997,760 bytes against a predicted $800,000 + 196,608 = 996,608$, so the model is short by 0.12 percent, which is the file header. HNSW serializes to 172.4442 MB, an overhead of 272.1 bytes per vector against the 256 predicted by the adjacency lists alone, the remaining 16 bytes being the level and offset arrays. The same 272.1 bytes per vector appears at $N = 20,000$, so the overhead is linear in N as the model requires. This matters for capacity planning more than it might seem: at $M = 32$ the HNSW graph adds 18 percent to the footprint of the raw vectors, and unlike the vectors it cannot be quantized away without changing the algorithm.

VII. SYSTEM IMPLEMENTATION

A. Modules and the index interface

The framework is about 2,800 lines of Python across seven library modules, three command-line entry points and three test modules. The piece that holds it together is a single abstract interface. Every index, exact or approximate, exposes the same two operations and the same build statistics, which is what allows the query engine and the evaluator to be written once and used three times.

```
class BaseIndex:
    def build(self, X)          -> BuildStats
    def search(self, Q, k)      -> (dists, ids)
```

```
BuildStats = (build_time_s, memory_bytes, n_vectors)
```

The orchestrator is correspondingly short. It materialises the vectors once, computes the ground truth once from a Flat index, and then loops over the configured methods. Nothing in this loop knows which index it is holding.

```
X, Q = materialise(cfg)          # ingest + embed, once
gt = Flat(d).build(X).search(Q, k) # exact reference
```

```
for m in cfg.methods:
    idx = build_index(m, d, cfg.params[m])
    bs = idx.build(X)
    res = run_queries(idx, Q, k, warmup)
    row = evaluate(bs, res, gt, k)
    db.log(run_id, 'INFO', 'query', row.summary())
    if row.recall < 0.95 and m != 'flat':
        db.log(run_id, 'WARN', 'evaluate', ...)
```

The warning on the last two lines is deliberate. An approximate index falling below the acceptance threshold is not an error and must not stop the run, but it is by far the most common reason a result looks wrong later, so it is written into the log where somebody reading the run will actually find it. Section IX describes the occasion when that warning did its job.

B. The results database

The framework writes CSV tables, and for a while that was enough. It stopped being enough once we started sweeping sizes and tuning parameters, because the questions changed shape. "What did this run report" is answered by a CSV. "How does HNSW at 100k compare with the sweep we ran last week, on the same machine, at the same ef_search " is answered by re-reading a directory of files by hand, or by a WHERE clause. We added the second option.

The schema has four tables. A run row holds the experiment type, the configuration verbatim, and the host: hostname, platform, Python version, FAISS version and core count. Those environment columns are not decoration, because the central claim of the project only holds for two rows produced on the same machine with the same libraries, and recording them makes that condition checkable rather than remembered. A measurement row is one MethodMetrics record, one per method per dataset size. A tuning_point row is one setting tried in a sweep, kept separate because a query-time knob varies on an index built once and so has no build time or memory of its own. A run_log row is one log line.

The integrity constraints do real work. Recall is a fraction by definition, so the schema CHECKs it into [0, 1]; a value outside that interval is not an unusual result but a broken evaluator, and the cheapest place to catch it is at write time next to the code that produced it, rather than three weeks later as an odd point on a chart. The method column is restricted to the three implemented families, UNIQUE (run_id, method, n_vectors) stops a re-run silently doubling up, and every child table cascades on delete so a discarded run cannot leave orphaned measurements behind. Four views sit on top for the joins we found ourselves writing repeatedly. The read side is a command-line client that lists, filters, sorts, exports and accepts ad-hoc SELECT statements; it refuses anything that is not a read, because a recorded measurement is evidence and evidence should not be editable from a convenience tool.

C. Environment and continuous integration

The implementation is Python 3.10 or newer, using FAISS for the indexes, sentence-transformers for embeddings, NumPy and pandas for data handling, matplotlib for charts, and SQLite from the standard library for persistence. The real-data results in this paper were produced on a 12-core x86-64 machine with 16 GB of RAM running Windows 11, with FAISS 1.14.3 and Python 3.13.1; the 100,000-vector sweep peaks a little above 1 GB.

Every push runs the test suite, the linter and a real end-to-end benchmark including the database write and read-back, on six clean runners: Ubuntu and Windows across Python 3.10, 3.11 and 3.13. The suite is 43 tests covering ingestion, embedding determinism, all three index families, the small-N nlist clamp, the evaluation metrics and the database constraints. We built this because the project claims reproducibility, and a framework that only runs on the machine of whoever wrote the module is not evidence for that claim.

VIII. EXPERIMENTAL EVALUATION

A. Setup

Two corpora are used. The real corpus is a 20,000-headline deterministic sample of A Million News Headlines [10], a CCO collection of ABC News headlines published between 2003 and 2021, de-duplicated and sampled at a fixed seed from the 1,244,184-row original. Headlines are short, self-contained and topically diverse, which is close to the shape of the documents a semantic search backend actually indexes. The synthetic corpus is 64 Gaussian cluster centres in 384 dimensions with points drawn at $\sigma = 0.15$ and L2-normalised, used for the scalability sweep so that N becomes a free parameter and the sweep measures index behaviour rather than embedding throughput. Both use $|Q| = 200$ held-out queries and $K = 10$.

B. Real-data comparison

Table I and Fig. 2 give the comparison on the real corpus. Flat is the exact reference at recall 1.000. HNSW is the fastest at 0.451 ms, four times quicker than Flat, and holds recall at 0.995; it pays for that with the highest build time at 2.73 s and the largest footprint at 34.5 MB. IVF sits between the two, 1.6 times faster than Flat at recall 0.963, with build time and memory close to Flat's. The tail behaviour is worth reading alongside the means. HNSW's p95 of 0.850 ms is 1.9 times its own mean, while IVF's p95 of 2.110 ms is 1.8 times its mean and only 6 percent better than Flat's p95 of 2.252 ms. IVF's advantage over exact search is therefore far weaker at the tail than its mean suggests, which is exactly the kind of thing a batch-timed benchmark cannot show.

TABLE I. PER-METHOD COMPARISON AT 20,000 VECTORS

Method	Configuration	Latency mean / p95 (ms)	Recall@10	Build (s)
Flat	Flat	1.818 / 2.252	1.000	0.011
IVF	IVF(nlist=100, nprobe=32)	1.168 / 2.110	0.963	0.223
HNSW	HNSW(M=32, ef_search=64)	0.451 / 0.850	0.995	2.729

Vector Index Comparison (single scale)

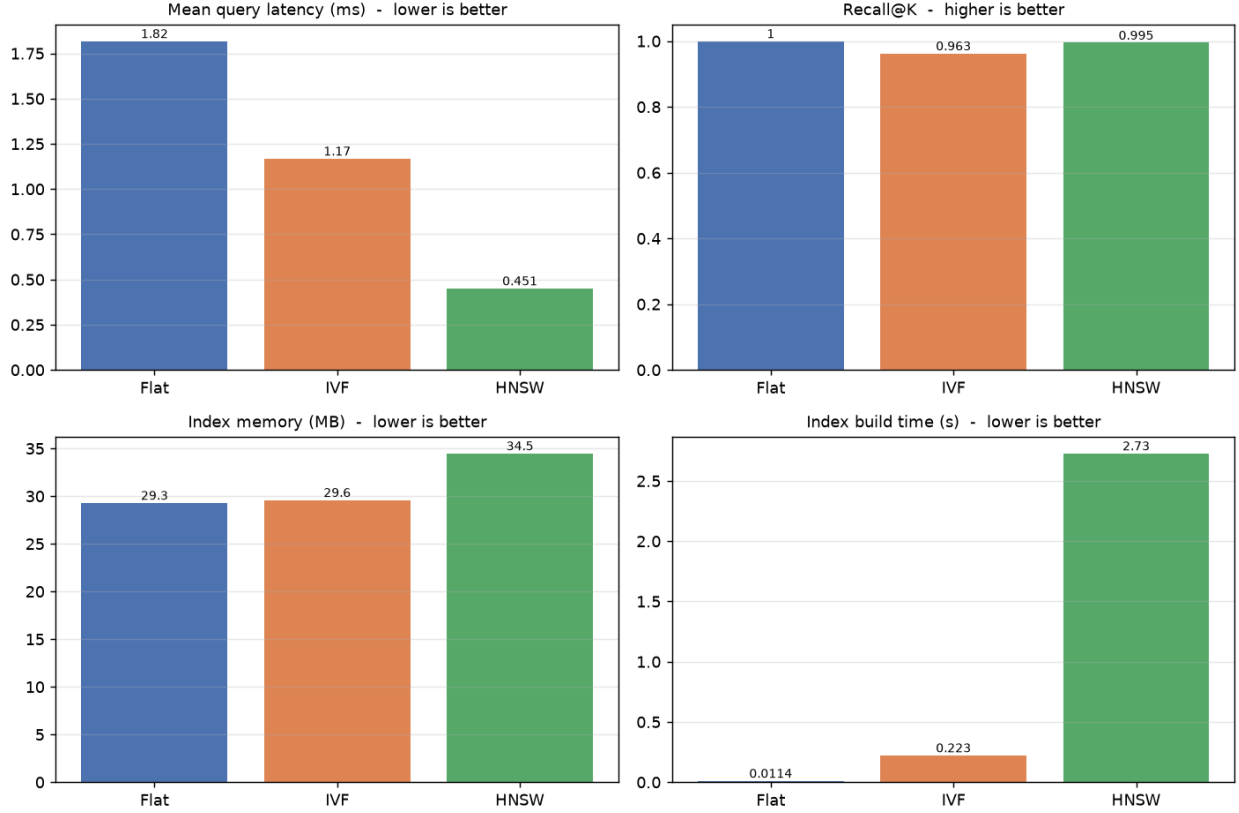


Fig. 2. Per-method comparison at 20,000 vectors: latency, recall@10, memory, and build time.

C. Scalability

Table II and Fig. 3 sweep all three methods from 1,000 to 100,000 vectors. Flat behaves as the cost model predicts, rising from 0.050 ms to 8.027 ms. IVF holds recall at 1.000 across the whole range and reaches 100,000 vectors at 1.017 ms. HNSW is the fastest method at every scale above 5,000 and ends at 0.338 ms, but its recall at a fixed `ef_search` of 64 falls steadily: 1.000, 0.995, 0.911, 0.803, 0.696.

That decay is the clearest result in the project and it follows from the cost analysis. `ef_search` bounds the size of the

candidate list the base-layer search is allowed to keep, and holding it constant while `N` grows means each query examines a smaller and smaller fraction of the graph. The speed advantage and the accuracy loss are the same phenomenon measured from two sides: over the sweep HNSW goes from being 0.9 times as fast as Flat at `N` = 1,000, where the graph descent is pure overhead on a tiny corpus, to 23.8 times faster at `N` = 100,000. An approximate index tuned once and left alone does not fail loudly as its corpus grows. It gets quietly less accurate, and nothing in the query path reports it.

TABLE II. SCALABILITY SWEEP, 1,000 TO 100,000 VECTORS

Metric	1,000	5,000	20,000	50,000	100,000
Flat latency (ms)	0.050	0.402	1.249	4.402	8.027
IVF latency (ms)	0.032	0.052	0.244	0.521	1.017
HNSW latency (ms)	0.057	0.055	0.153	0.264	0.338
HNSW recall@10	1.000	0.995	0.911	0.803	0.696

Scalability: metrics vs dataset size

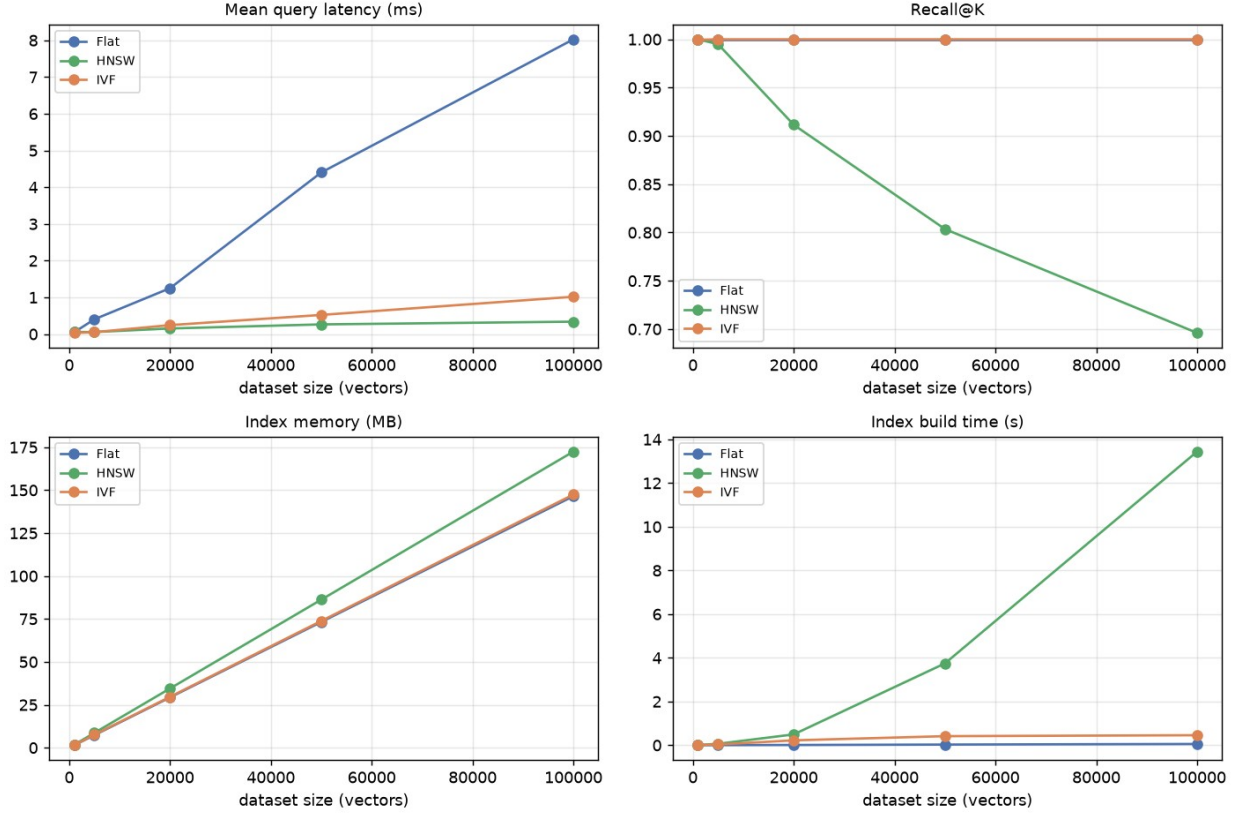


Fig. 3. Scalability sweep: latency, recall, memory, and build time versus dataset size.

D. Parameter tuning

The tuning sweep builds an index once and re-queries it across its query-time knob, since both `nprobe` and `ef_search` are query-time settings and rebuilding at each step would only waste time. Fig. 4 shows the two curves on the real corpus. IVF climbs from 0.506 at `nprobe` = 1 through 0.839 at 8 and 0.910 at 16 to 0.963 at 32, reaching 1.000 only at `nprobe` = 100, which is to say by searching every cell and becoming exact search with extra steps. HNSW starts at 0.977 with `ef_search` = 8 and is at 0.997 by 64; past that, 128 and 256 buy 0.002 of recall for double the latency.

Read against the constrained objective of Section II-B, the two methods reach the 0.95 floor at very different prices. HNSW clears it at its smallest tested setting, at 0.092 ms. IVF needs `nprobe` = 32, at 0.726 ms, roughly eight times the latency for a slightly lower recall. The shapes differ too: HNSW's curve is flat, so a mis-set `ef_search` costs little, while IVF's is steep between 1 and 16, so the same carelessness costs a great deal. For an operator, a flat curve is worth something on its own, independently of where it sits.

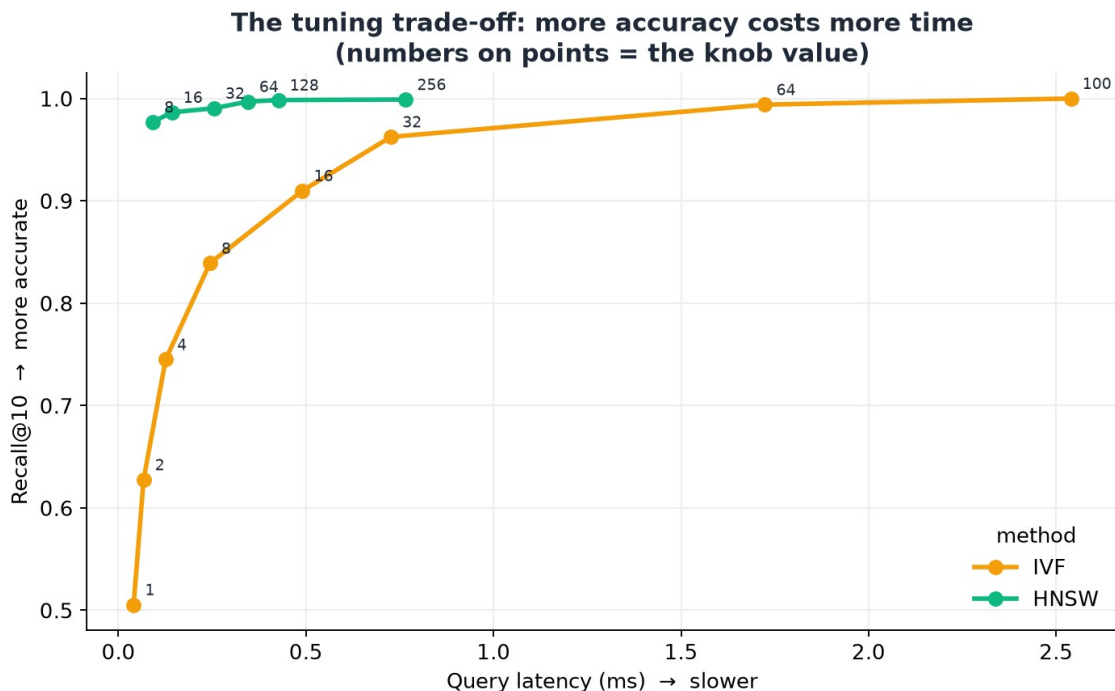


Fig. 4. Tuning trade-off: recall@10 against query latency for IVF (nprobe) and HNSW (ef_search); the label on each point is the knob value.

E. Reproducibility, and where it stops

Because reproducibility is the framework's central claim, we tested it rather than assumed it. Re-running a configuration on the same machine reproduces the corpus sample, the held-out queries and the embeddings byte for byte. Serialized index size is stable to the byte across machines: the HNSW index at $N = 20,000$ serializes to 36,162,530 bytes on both our Windows development machine and a Linux runner. Flat and IVF recall reproduce exactly.

HNSW recall does not, and we think this is worth reporting rather than hiding. Repeating the $N = 20,000$ synthetic build three times on one Linux machine, at a fixed seed, gave recall 0.889, 0.894 and 0.884; the same configuration on the Windows machine that produced Table II gave 0.911. The cause is that FAISS inserts points into the HNSW graph in parallel, so which neighbours a node ends up linked to depends on thread interleaving, and the graph therefore differs slightly from build to build even though its size does not. Forcing construction onto a single thread by setting `OMP_NUM_THREADS=1` removes the variation completely: three runs then gave 0.8985, 0.8985 and 0.8985.

The practical consequence is a distinction the framework had blurred and now states. Corpus sampling, embeddings, index size, and Flat and IVF recall are deterministic and can be compared across machines. HNSW recall carries a spread of roughly plus or minus 0.5 points at default thread settings, which is small against the 30-point decay in Section VIII-C but not small against the 0.2-point differences between adjacent `ef_search` values in Section VIII-D. Anyone using this framework to choose between two nearby HNSW settings should pin the thread count first.

IX. DISCUSSION

There is no universal winner, which is an unsatisfying result to report and the correct one. On real data both approximate methods come close to exact accuracy while being substantially faster, and the right choice depends on the workload rather than on the numbers alone. HNSW gives the lowest latency, the flattest tuning curve and near-constant query cost as the corpus grows, which suits a read-heavy service that can absorb a build cost measured in minutes and a footprint 18 percent above the raw vectors. IVF builds thirty times faster and holds recall perfectly on well-separated data, which suits a corpus that is re-indexed often. The scalability result sharpens the choice rather than settling it: HNSW's parameters have to be raised as the data grows, and the tuning sweep is what tells an operator by how much.

The project also caught us out, which we think is the strongest evidence that it does what it claims. IVF's defaults of `nlist = 100` and `nprobe = 8` reach recall 1.000 on the synthetic clustered vectors and had done so in every run since the midterm checkpoint. On real Sentence-BERT embeddings of news headlines the same settings reach 0.839, well under our 0.95 threshold. The reason is that real embeddings of short text overlap far more than clean Gaussian clusters, so the true neighbours of a query are scattered across many more cells than `nprobe = 8` will visit. The framework flagged it as a WARN in the run log, the tuning sweep quantified it, and the fix was to raise `nprobe` to 32. Tuning on synthetic data and deploying against real data is precisely the error the project was built to expose in other people's benchmarks, and both runs are still in the database, so the before and after is one query rather than a claim.

For a practitioner the results reduce to a short rule. If the corpus is stable and queries dominate, use HNSW, set `ef_search` from a sweep on your own embeddings rather than from a

tutorial, and re-run the sweep whenever the corpus grows by an order of magnitude. If the corpus is rebuilt frequently, or if memory is tight, use IVF and expect to search a larger fraction of the cells than any synthetic benchmark will suggest. Below roughly 10,000 vectors, use exact search: at $N = 1,000$ Flat beat HNSW outright in our sweep, and an approximate index there buys nothing but a parameter to get wrong.

X. THREATS TO VALIDITY

Latency is a wall-clock measurement and moves with machine load. We mitigate this with warm-up queries and per-query timing, and we only compare latencies between runs on the same host. Section VIII-E sets out the corresponding limit on recall: it is deterministic for Flat and IVF but not for HNSW at default thread settings, so HNSW recall differences smaller than about half a point should not be read as real.

The scalability sweep uses synthetic vectors, whose clean cluster structure flatters IVF relative to real embeddings, as Section IX describes in detail. We state this rather than smoothing over it, and the accuracy claims in the paper rest on the real-data benchmark. The real corpus is itself narrow: news headlines are short, English and single-domain, and results on long documents or multilingual text could differ. Finally the study is single-machine and CPU-only by scope, so nothing here should be read as a claim about distributed sharding, GPU-accelerated search, or index behaviour under concurrent load.

XI. CONCLUSION AND FUTURE WORK

We built a benchmarking framework that compares Flat, IVF and HNSW under identical conditions and reports latency percentiles, recall against exact ground truth, memory, build time and scalability from a single command. Every figure and table in this paper comes from artefacts committed to the repository, and every run is recorded in the results database with the configuration and the machine that produced it. The main empirical finding is that an approximate index's parameters are a function of corpus size rather than a one-time setting, and that HNSW in particular loses accuracy silently as data grows. The

main methodological finding is smaller but was more useful to us: recording the host environment alongside the numbers is what turned an anomalous IVF result into something we could diagnose.

Three directions follow naturally. The index interface is narrow enough that Product Quantization, ScaNN [9] or DiskANN could be added without touching the rest of the pipeline, which would extend the comparison to the compressed and disk-resident families this work does not cover. The corpus could be broadened beyond short English news text to long-form documents and to multilingual or multimodal embeddings. And since the framework already measures how recall decays with N , the sweep could be inverted into a scheduler: given a target recall and a growth rate, solve for when `ef_search` must next be raised, and have the system say so rather than wait to be asked.

XII. TEAM INVOLVEMENT AND PROJECT ORGANIZATION

Four members worked on the project across the full term. Rather than share a pool of tasks we used a four-lane ownership model, in which each member owns a disjoint set of modules. Two people never edit the same file, merge conflicts were close to non-existent, and the commit history is itself the record of who did what, so no contribution claim has to be taken on trust. Because the lanes follow the architecture, a lane boundary is also an interface boundary, and that is what made honest progress reporting possible at the midterm.

Work was tracked on a GitHub Projects board with one column per state and one label per lane, at the granularity of the sixteen-task breakdown from the D.3.1 status report, with each issue closed by the commit that implements it. Two conventions survived the whole term. Nothing was pushed until it ran, which is why the midterm report said HNSW was in progress rather than claiming it was done. And an experiment is recorded by its config, not by its code, which is what makes every number in this paper traceable to a file. All sixteen planned tasks are complete.

TABLE III. INDIVIDUAL CONTRIBUTIONS BY LANE

Member	Contribution
Devu Babu Sheeja	Dataset selection and justification; ingestion, cleaning, and segmentation; the deterministic Sentence-BERT and offline embedding backends (<code>data_ingestion.py</code> , <code>prepare_headlines.py</code> , <code>embeddings.py</code> , <code>fetch_dataset.py</code>).
Ashwin Senthur Pandian	The indexing layer: Flat, IVF with adaptive nlist scaling, and HNSW, plus the parameter-tuning sweep (<code>indexing.py</code> , <code>tune_parameters.py</code>).
Rishi G. Patel	The query engine and evaluation logic (timing, <code>recall@K</code> , orchestration) and the results database and its integrity constraints (<code>query_engine.py</code> , <code>benchmark.py</code> , <code>results_db.py</code> , <code>db/schema.sql</code>).
Nikhil Goud Nathi	The benchmark CLI, configuration system, scalability experiments, visualization, results-database CLI, the shared repository, and the continuous-integration workflow (<code>run_benchmark.py</code> , <code>visualization.py</code> , <code>db_cli.py</code> , <code>configs/</code>).

ACKNOWLEDGMENT

We thank Dr. Andreas S. Maniatis for guidance on project scope and evaluation methodology through the proposal and midterm reviews. We also acknowledge the Australian Broadcasting Corporation and the maintainers of A Million News Headlines for publishing the corpus under a CC0 dedication, which is what allowed us to commit a sample of it

alongside the code and make the real-data results reproducible by anyone.

REFERENCES

- [1] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

- [2] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [4] H. Jégou, M. Douze, and C. Schmid, “Product Quantization for Nearest Neighbor Search,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, Jan. 2011.
- [5] M. Aumüller, E. Bernhardsson, and A. Faithfull, “ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms,” *Information Systems*, vol. 87, 101374, 2020.
- [6] J. Wang et al., “Milvus: A Purpose-Built Vector Data Management System,” in *Proc. ACM SIGMOD*, 2021, pp. 2614–2627.
- [7] P. Indyk and R. Motwani, “Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality,” in *Proc. 30th ACM Symp. Theory of Computing (STOC)*, 1998, pp. 604–613.
- [8] M. Douze et al., “The Faiss Library,” *arXiv:2401.08281*, 2024.
- [9] R. Guo et al., “Accelerating Large-Scale Inference with Anisotropic Vector Quantization,” in *Proc. 37th Int. Conf. Machine Learning (ICML)*, 2020, pp. 3887–3896.
- [10] R. Kulkarni, “A Million News Headlines,” *Harvard Dataverse*, V11, doi:10.7910/DVN/SYBGZL, CC0 1.0. Experimental corpus for this study.
- [11] D. R. Hipp, “SQLite,” version 3, <https://www.sqlite.org>. Embedded relational engine used for the results database.
- [12] J. J. Pan, J. Wang, and G. Li, “Survey of Vector Database Management Systems,” *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, 2024.

ADDENDUM: REPOSITORY CONTENTS

The complete application, dataset sample, database schema, populated results database, configuration files, test suite and continuous-integration workflow are in the project repository at github.com/Devz0201/COMP8157-Vector-Indexing, together with the D.4.1 requirements, D.4.2 design, D.4.4 deployment and D.4.5 user guide documents under docs/. The four YAML files under configs/ reproduce every figure and table above: scalability.yaml for Table II and Fig. 3, text_sbert.yaml for Table I and Fig. 2, tuning.yaml for Fig. 4, and ci_smoke.yaml for a ten-second installation check. The populated database at db/vecbench_results.db holds the four runs those figures came from, each with the configuration and host machine that produced it.